Stage/week 3

Floating-Point Divider Implementation in Verilog Using Newton-Raphson's Approximation

Group 14

EDA playground link

Work done in this stage:

In Stage 2 the top-level FSM and the module interfaces were complete, but the multiplier and the subtractor were behavioural placeholders (a * b and 2 - e). In Stage 3 we replaced both with structural designs built from gates and smaller blocks. The ports are unchanged, so the FSM needed no modification.

- Vedic multiplier (Urdhva Tiryagbhyam), built hierarchically from a gate-level 2x2 block up to 32x32, plus a 24x24 version for the IEEE-754 mantissa.
- 32-bit subtractor built on a hybrid carry look-ahead and ripple carry adder: four 8-bit CLA blocks, with the carry rippling between the blocks.
- Integration of both modules into the divider datapath (mant_mul and the 2 - e step of the Newton-Raphson loop), with a separate self-checking testbench for each module.

New and modified files:

File	Status	Description
vedic_mul.v	New	ha, vedic_2x2, vedic_4x4, vedic_8x8, vedic_16x16, vedic_32x32, vedic_24x24
full_adder.v	New	1-bit gate-level full adder
cla8.v	New	8-bit carry look-ahead adder
sub32.v	New	32-bit subtractor (four cla8 blocks, ripple between blocks)
mant_mul.v	Modified	Now instantiates vedic_32x32 (parameter USE_VEDIC)
fp_div.v	Modified	2 - e now computed by sub32
tb_vedic.v, tb_sub32.v	New	Module-level self-checking testbenches

Vedic multiplier

Principle

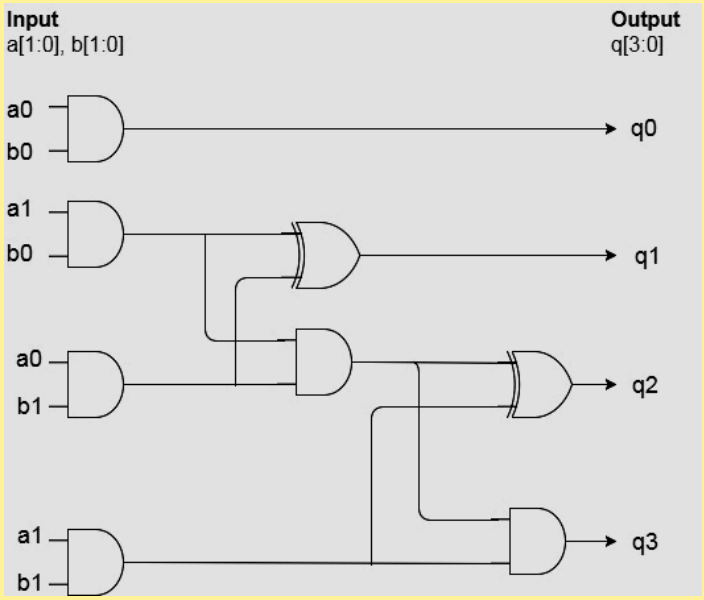
The Urdhva Tiryagbhyam sutra ("vertically and crosswise") forms all partial products in parallel and adds them column by column, instead of shifting and adding row by row as an array multiplier does. This shortens the carry chain compared with the array multiplier we considered in Stage 2.

Base case: 2x2 block (gate level)

For a = a1a0 and b = b1b0:

- p0 = a0.b0 (vertical)
- p1 = (a1.b0) ⊕ (a0.b1), with carry c1 = (a1.b0).(a0.b1) (crosswise)
- p2 = (a1.b1) ⊕ c1 and p3 = (a1.b1).c1 (vertical plus carry)

The block uses 4 AND gates and 2 half adders.



(Referenced from **A Novel Efficient Hardware Implementation of Elliptic Curve Cryptography Scalar Multiplication Using Vedic Multiplier**)

Hierarchical extension (4x4 to 32x32)

Each nxn block splits both operands into high and low halves of n/2 bits and uses four n/2 x n/2 blocks:

- q0 = aL.bL (vertical)
- q1 = aH.bL and q2 = aL.bH (crosswise)
- q3 = aH.bH (vertical)

The partial products are combined as P = q0 + ((q1 + q2) << n/2) + (q3 << n). The 4x4 block uses four 2x2 blocks, the 8x8 uses four 4x4 blocks, and so on up to 32x32.

24x24 mantissa multiplier

The IEEE-754 single-precision significand is 24 bits including the hidden 1, which is not a power of two. So vedic_24x24 treats each operand as three 8-bit digits {H, M, L} and uses nine vedic_8x8 blocks, combining them column by column like the hand multiplication of two 3-digit numbers:

- col0 = L.L
- col1 = M.L + L.M
- col2 = H.L + M.M + L.H
- col3 = H.M + M.H
- col4 = H.H

P = col0 + (col1 << 8) + (col2 << 16) + (col3 << 24) + (col4 << 32)

Integration into the divider

mant_mul is the single shared multiplier used by every step of the divider (e = d.x, x.(2 - e) and A.x). It now instantiates vedic_32x32. A parameter USE_VEDIC (default 1) keeps the behavioural * selectable for comparison, and the ports are the same as in Stage 2.

Subtractor with hybrid CLA and ripple carry adder

Why a hybrid adder

- A ripple carry adder is small, but every carry waits for the previous one, so the delay grows linearly with the width (32 full-adder delays for 32 bits).
- A full 32-bit CLA is fast, but the carry equations for the higher bits need very wide AND and OR gates.
- The hybrid design uses look-ahead inside each 8-bit block and ripples the carry between the four blocks. The longest carry path is then about 4 block delays instead of 32 bit delays, and the gates stay a reasonable size.

8-bit CLA block (cla8)

- Generate $g_i = a_i \cdot b_i$ and propagate $p_i = a_i \oplus b_i$
- Each carry is computed directly from g, p and cin, so no carry waits for the previous one: $c_{i+1} = g_i + p_i g_{i-1} + p_i p_{i-1} g_{i-2} + \ldots + p_i \ldots p_0 .\text{cin}$
- Sum $s_i = p_i \oplus c_i$

32-bit subtractor (sub32)

Subtraction uses two's complement: a - b = a + (~b) + 1.

- b is inverted bit by bit before it enters the adder.
- The +1 is applied as the carry-in of the first CLA block ($c_0 = 1$).
- The carry out of each block is the carry in of the next block.
- borrow = NOT(carry out of bit 31), so borrow = 1 when a < b.

Use in the Newton–Raphson loop

In fp_div, the 2 - e term of x(n+1) = x(n).(2 - d.x(n)) is now computed by sub32, with a = 2.0 (Q2.30, 32'h8000_0000) and b = e. In Stage 2 this was a behavioural subtraction.

Verification

All simulations were run in Icarus Verilog.

Module-level testbenches

Each module was checked against the behavioural * and - operators:

Module	**Test cases**	**Mismatches**
vedic_8x8	65,536 (exhaustive, all input pairs)	0
vedic_24x24	20,004 (4 corner + 20,000 random)	0
vedic_32x32	20,004 (4 corner + 20,000 random)	0
sub32	20,012 (12 corner + 20,000 random)	0

The corner cases include zero, all ones, single-bit values, a < b, a = b, and 2 - e with e ≈ 1.0, which is the case that occurs in the Newton-Raphson loop.

Full divider with the new modules

The complete divider, now using vedic_32x32 and sub32, was run on 4,000 test vectors generated by the C++ golden model:

NR iterations	**Latency (cycles)**	**Exact**	**Within 1 ULP**	**Max ULP error**

0 (LUT seed only)	4	0 %	0 %	986,895
1	6	0.33 %	0.60 %	58,053
2	8	31.88 %	45.13 %	201
3	10	99.53 %	100 %	1

After 3 iterations every result is exact or within 1 ULP. The maximum error falls from 986,895 ULP (LUT seed only) to 58,053, 201 and 1 ULP after 1, 2 and 3 iterations, which matches the quadratic convergence of Newton–Raphson. Running the same vectors with the behavioural * (USE_VEDIC = 0) gives identical results, so the Vedic multiplier introduces no error.

Problems encountered

1. Extending the Vedic multiplier to 24 bits: 24 is not a power of two, so the 2-way split could not be reused directly. We solved this with the 3-digit (8-bit) column-wise version.
2. Getting the carry-in and borrow right for two's complement subtraction (the +1 as c_0 and borrow = NOT cout).
3. Choosing a block size for the hybrid adder that balances speed against gate size.

Current status and next steps

Done: gate-level Vedic multiplier and hybrid CLA/ripple subtractor, both integrated into the divider and verified.
Remaining:

1. Replace the behavioural + that combines the partial products inside the Vedic blocks with the CLA adders.
2. Wire vedic_24x24 into a dedicated floating-point mantissa multiplier.
3. Handle the IEEE-754 special cases (zero, infinity, NaN, subnormals) and overflow and underflow at the output.
4. Measure latency and accuracy against the iteration count for the final report.

Workload division

- Gurnoor: Design and implementation of the Vedic multiplier using the Urdhva Tiryagbhyam method, including hierarchical 2x2, 4x4, 8x8, 16x16 and 32x32 multiplier blocks,along with the dedicated 24×24 Vedic multiplier for floating-point significand multiplication.

- Keerat: Design and implementation of the FSM integration and datapath coordination within fp_div , including verifying state transitions, checking control signals, coordinating

the shared multiplier and subtractor operations, and debugging the interaction between the FSM and arithmetic modules.

- Monish: Integration of the Vedic multiplier and subtractor into the existing floating-point divider datapath, along with the design and implementation of the FSM using local parameters (IDLE, S_E, S_X, S_Q, and S_OUT). This includes controlling Newton-Raphson iterations, managing shared multiplier connections, sequencing arithmetic operations, and coordinating datapath control signals.

- Divya: Design and implementation of the hybrid 32-bit subtractor using four 8-bit carry look-ahead adder (CLA) blocks, including carry propagation between blocks and borrow generation.

- Mansi: Implementation of IEEE-754 special-case handling, including NaN, infinity and related exceptional input combinations, along with output handling for special results and verification of these cases.

References

1. From Baseline to High Throughput: Architecture and Resource Co-Optimization of a Verilog-Based Newton–Raphson Divider on FPGAs, Wentao Ding (gave us the idea to reuse the same multiplier multiple times with a finite state machine)
2. C. S. Aragula, B. Preethi and C. Ramesh, "Design and Implementation of an Efficient 32-Bit Fixed-Point Newton-Raphson Division-Based Reciprocal Computing Unit"(helped us understand the newton raphson method)
3. Naginder Singh and Trailokya Nath Sasamal, "Design and Synthesis of Single Precision Floating Point Division based on Newton-Raphson Algorithm on FPGA"(packing and unpacking of IEEE 754 numbers as well as the idea to use vedic multipliers in the multiplication module)
4. "A Novel Time- Area-Power Efficient Single Precision Floating Point Multiplier", Himanshu Thapliyal and M.B Srinivas, International Institute of Information Technology (helped when constructing the 24x24 vedic multiplier which had the bottleneck where 24 wasn't a power of 2)
5. https://www.youtube.com/watch?v=YtNrObZNiDA (helped us understand the method behind the vedic multiplier)
6. https://github.com/tapeoutcutm/High-Speed-4x4-Vedic-Multiplier-for-Efficient-Arithmetic-Operations(helped in debugging our vedic multiplier)